

UrlParameterParser

Gegeben ist folgender **Unit Test** für die Klasse **UrlParameterParser**, der **aus einer URL** alle **Parameter als Map** extrahiert. **Implementiere** die zugehörige Klasse **UrlParameterParser**!

Hinweise:

Parameter **starten nach** dem **ersten ?** in der URL und werden **anschließend** mit **&** **getrennt**.

Parameter haben immer einen **Key** und einen **Value**. Key und Value werden mit einem **=** **getrennt**.

Beispiel: `https://haw-hamburg.de?parameter1=value1¶meter2=value2¶meter3=value3`

Test zeigen

39

Bevor wir implementieren, schauen wir uns die 'Verträge' an, die wir erfüllen müssen. Wir haben eine Testklasse `UrlParameterParserTest` erhalten. Diese Tests definieren genau, wie sich unser Parser verhalten soll. Gehen wir sie kurz durch, damit wir wissen, was unser Regex können muss:

testUrlWithMultipleParameters: Das ist unser 'Happy Path'. Eine Standard-URL mit ? und Parametern, die durch & getrennt sind. Wir erwarten eine Map mit allen Paaren.

testUrlWithNoParameters: Ein wichtiger Randfall. Wenn kein ? da ist, darf nichts abstürzen. Wir erwarten einfach eine leere Map.

testUrlWithSpecialCharacters: Hier sehen wir, dass Werte auch Sonderzeichen wie % enthalten können. Unser Regex darf also beim Value nicht zu strikt sein (z.B. nur Buchstaben), sondern muss fast alles akzeptieren, was kein Trennzeichen ist.

testUrlWithIncompleteParameters: Spannender Fall: `different=` endet direkt. Das bedeutet, der Key ist da, aber der Value ist leer. Unser Regex muss also auch leere Werte akzeptieren (Stern * statt Plus +).

testUrlWithInvalidFormat: Hier steht `?param1` ohne ein Gleichheitszeichen. Der Test erwartet eine leere Map. Das heißt, unser Parser soll nur strikte Key=Value

Paare akzeptieren und alles andere ignorieren."

```
public class UrlParameterParser {  
    public static Map<String, String> parse(String url) {  
        Map<String, String> params = new HashMap<>();  
  
        // TODO  
  
        return params;  
    }  
}
```

40

Okay, die Tests sind rot (würden fehlschlagen). Lasst uns die Klasse UrlParameterParser bauen, um sie grün zu bekommen.

Wir starten mit einer leeren HashMap. Damit erfüllen wir schon fast testUrlWithNoParameters und testUrlWithInvalidFormat, weil wir zumindest eine leere Map zurückgeben und keine NullPointerException werfen.

Warum Hashmap?

41

1. Die Natur der Daten (Schlüssel-Wert-Paare)

URL-Parameter sind klassische **Key-Value-Paare**. Ein Parameter besteht immer aus einem Namen (z. B. param1) und einem Wert (z. B. value1).

Eine List oder ein Array wäre hier unpraktisch, da wir nur eine Liste von Strings hätten (z. B. ["param1", "value1", ...]) und wissen müssten, welcher String zu welchem gehört.

Eine Map bildet genau diese logische Struktur ab: Ein Schlüssel zeigt auf einen Wert.

2. Performance (Schneller Zugriff)

Der Hauptzweck des Parsens ist es meistens, später gezielt auf einen bestimmten Parameter zuzugreifen.

Schau dir den Test an: parameters.get("param1").

Bei einer HashMap ist dieser Zugriff **extrem schnell** (in der Informatik spricht man von $O(1)$ bzw. konstanter Zeit). Wir müssen nicht durch eine Liste suchen ("Ist das param1? Nein. Ist das param1? ..."), sondern die Map springt direkt zum Wert.

3. Eindeutigkeit der Schlüssel

In einer Map darf jeder Schlüssel (Key) nur **einmal** vorkommen.

Das passt perfekt zur Standard-Logik von URL-Parametern: Wenn id=5 in der URL

steht, wollen wir unter dem Schlüssel "id" genau diesen Wert finden. Würden wir eine List verwenden, könnten wir theoretisch fünfmal "id" speichern. Die HashMap überschreibt bei einem doppelten Schlüssel automatisch den alten Wert (oder behält ihn, je nach Implementierung), garantiert aber, dass der Key eindeutig bleibt.

4. Interface vs. Implementierung (Java Basics)

Die Methodensignatur verlangt als Rückgabotyp eine Map.

Map ist in Java aber nur ein **Interface** (eine Art Vertrag). Man kann kein new Map() schreiben.

Wir brauchen eine **konkrete Klasse**, die diesen Vertrag erfüllt. Die HashMap ist die Standard-Implementierung in Java für diesen Zweck.

Zusatz für Profis (optional): "Wenn euch die **Reihenfolge** der Parameter wichtig ist (also dass param1 vor param2 in der Map steht), müsstet ihr eine LinkedHashMap nehmen. Einer normalen HashMap ist die Reihenfolge egal – sie speichert die Daten so, wie es für sie am effizientesten ist."

```
public static Map<String, String> parse(String url) {  
    Map<String, String> params = new HashMap<>();  
  
    // 1. Alles hinter dem ? finden  
    Pattern queryPattern = Pattern.compile("\\?(.*)$");
```

42

Sprechertext: "Schauen wir auf testUrlWithMultipleParameters. Die Parameter stehen hinter dem ?. Wir müssen also erst prüfen: Gibt es ein Fragezeichen? Und wenn ja, holen wir uns alles dahinter.

Hier definieren wir das Suchmuster. Das Pattern-Objekt ist quasi die **kompilierte Schablone** für unseren Regex. Wir kompilieren es, damit die Regex-Engine den String nicht jedes Mal neu "verstehen" muss, wenn wir ihn benutzen.

\\? (Das Fragezeichen):

Das Fragezeichen ? ist in Regex eigentlich ein **Quantifizierer** (bedeutet "0 oder 1 mal").

Wir wollen hier aber nach einem **echten** Fragezeichen-Zeichen suchen (dem Start des Query-Strings).

Deshalb müssen wir es "escapen" (entwerten). In Regex wäre das \?.

Java-Besonderheit: Da der Backslash \ auch in Java-Strings ein Sonderzeichen ist, müssen wir ihn verdoppeln. Aus \ wird \\. Deshalb schreiben wir \\?.

(...) (Die Gruppe):

Die runden Klammern erzeugen eine **Capturing Group** (Fang-Gruppe).

Das bedeutet: "Suche das Muster darin, aber **merk dir** genau diesen Teil separat, damit wir ihn später herausholen können."

. * (Der Inhalt):

Der Punkt . steht für jedes beliebige Zeichen.

Der Stern * steht für "beliebig oft".

Zusammen heißt (. *): "Merk dir alles, was hier steht."

\$ (Das Ende):

Das ist der Anker für das Zeilenende. Es stellt sicher, dass wir wirklich bis zum allerletzten Zeichen der URL lesen.


```
public static Map<String, String> parse(String url) {  
    Map<String, String> params = new HashMap<>();  
  
    // 1. Alles hinter dem ? finden  
    Pattern queryPattern = Pattern.compile("\\?(.*)$");  
    Matcher queryMatcher = queryPattern.matcher(url);
```

43

Das Pattern ist nur der abstrakte Plan.

Der Matcher ist der **Arbeiter**, der diesen Plan auf einen ganz konkreten Text (unsere url) anwendet. Er speichert den aktuellen Zustand: Wo bin ich im Text? Habe ich was gefunden?

```

public static Map<String, String> parse(String url) {
    Map<String, String> params = new HashMap<>();

    // 1. Alles hinter dem ? finden
    Pattern queryPattern = Pattern.compile("\\?(.*)$");
    Matcher queryMatcher = queryPattern.matcher(url);

    if (queryMatcher.find()) {
        String queryString = queryMatcher.group(1);
        // Jetzt haben wir z.B. "param1=value1&param2=value2"

        // TODO: Parameter zerlegen
    }
}

```

44

Die Methode find() gibt den Startschuss. Der Matcher scannt den Text von links nach rechts.

Er sucht nach der ersten Stelle, die auf unser Muster \?(.*)\$ passt.

Wichtig: Wenn die URL kein Fragezeichen enthält, gibt find() einfach false zurück. Der if-Block wird übersprungen und wir vermeiden einen Fehler (NullPointerException oder ähnliches).

Zweite Zeile

Hier passiert die Magie der runden Klammern. Der Matcher hält das Ergebnis in "Gruppen" bereit:

group(0): Das ist immer der **gesamte Treffer**.

In unserem Fall wäre das: ?param1=value1. (Also *inklusive* dem Fragezeichen, denn das war ja Teil des Musters \?).

group(1): Das ist der Inhalt der **ersten Klammer** (.*)

Das ist genau der Teil *nach* dem Fragezeichen: param1=value1.

Warum machen wir das? Weil wir das Fragezeichen zum Suchen brauchten (als Anker), es aber im Ergebnis nicht haben wollen. Mit group(1) schneiden wir es elegant ab, ohne substring berechnen zu müssen.

```

public static Map<String, String> parse(String url) {
    Map<String, String> params = new HashMap<>();

    // 1. Alles hinter dem ? finden
    Pattern queryPattern = Pattern.compile("\\?(.*)$");
    Matcher queryMatcher = queryPattern.matcher(url);

    if (queryMatcher.find()) {
        String queryString = queryMatcher.group(1);

        // 2. Key-Value Paare finden
        // Gruppe 1: Key (mindestens 1 Zeichen, kein & oder =)
        // Gruppe 2: Value (beliebig viele Zeichen, kein & oder =)
        Pattern paramPattern = Pattern.compile("([^&=]+)=([^&=]*)");
        Matcher paramMatcher = paramPattern.matcher(queryString);

        // TODO: Durchlaufen
    }

    return params;
}

```

45

Sprechertext: "Jetzt haben wir den String param1=value1¶m2=value2. Wir könnten split('&') nehmen, aber schauen wir auf testUrlWithIncompleteParameters (different=) und testUrlWithInvalidFormat (param1). Ein Regex ist hier robuster, um sicherzustellen, dass wir wirklich Paare haben.

Wir definieren ein Paar so:

Key: Zeichen, die *kein* = und *kein* & sind. Mindestens eins. -> `[^&=]+`

Trennzeichen: Ein = muss zwingend da sein. (Das löst den Testfall testUrlWithInvalidFormat, wo das = fehlt).

Value: Zeichen, die *kein* = und *kein* & sind. Aber Achtung bei

testUrlWithIncompleteParameters: Der Wert kann leer sein! Also nehmen wir den Stern * statt Plus. -> `[^&=]*`

Zusammengesetzt: `([^&=]+)=([^&=]*)`

```

if (queryMatcher.find()) {
    String queryString = queryMatcher.group(1);

    // 2. Key-Value Paare finden
    // Gruppe 1: Key (mindestens 1 Zeichen, kein & oder =)
    // Gruppe 2: Value (beliebig viele Zeichen, kein & oder =)
    Pattern paramPattern = Pattern.compile("[^&=]+(=[^&=]*)");
    Matcher paramMatcher = paramPattern.matcher(queryString);

    while (paramMatcher.find()) {
        String key = paramMatcher.group(1);
        String value = paramMatcher.group(2);
        params.put(key, value);
    }
}

```

46

Der letzte Schritt. Wir nutzen `while(matcher.find())`, um durch den String zu 'wandern'. Jedes Mal, wenn das Muster passt, holen wir uns Gruppe 1 (Key) und Gruppe 2 (Value) und packen sie in die Map. Da unser Regex explizit `...=...` verlangt, werden ungültige Teile wie `param1` (ohne Ist-Gleich) automatisch übersprungen. Das macht den Test `testUrlWithInvalidFormat` grün.

Finale Zusammenfassung (Code Review gegen Tests)

Sprechertext: "Schauen wir uns das Ergebnis an.

Multiple Params? Ja, die while-Schleife holt alle.

No Params? Ja, `queryMatcher.find()` schlägt fehl, wir geben leere Map zurück.

Special Chars? Ja, `[^&=]*` erlaubt alles außer den Trennzeichen, auch %.

Incomplete (key=)? Ja, der Stern `*` beim Value erlaubt leere Strings.

Invalid (key)? Ja, das Regex verlangt zwingend ein `=`, sonst matcht es nicht.

Damit sollten alle Tests grün sein!"

Fazit

47

Regex ist formalisierte Sprache. LLMs sind exakt für so etwas gemacht. Die ganze aktuelle AI Welt basiert zu großen Teilen auf dem Verständnis von menschlicher Sprache. Übersetzer, Grammatik usw. Dafür ist es gemacht. Kein Wunder, dass diese LLMs extrem gut darin sind Reguläre Ausdrücke zu schreiben. Und dafür braucht es anders als Gemini, Chat GPT oder so garnicht viel Leistung. Viele Systeme nutzen kleine spezialisierte LLMs für konkrete Aufgaben. Bei aller Kritik an den großen/schweren/hungrigen und vor allem “alles könnenden“ Als. Bei Sprache sind sie wirklich gut, wenn man sie spezifisch und in den meisten Fällen auch lokal einsetzt.

Wie unsere Mehrkernigkeit: Lohnt sich der Aufwand für die Aufgabe? Ich würde sagen an dieser Stelle ist die Antwort häufig ja.